PROJECT OVERVIEW

Portunus

A self-hosted, zero-knowledge secrets manager backed by your own Google Drive

Your vault is encrypted in the browser and stored as a single file in your Drive. The server has no database, never sees plaintext, and ships as one Docker image.

Zero-knowledge | Password + keyfile | Stateless server | Single Docker image | Personal vault, not a team tool

Named after the Roman god of keys, doors and harbours · github.com/ralphv/portunus

THE IDEA

Three design pillars

Every other part of the project follows from these three choices.

Zero-knowledge

- Encryption and decryption happen in the browser
- The server never receives plaintext, the master password, the keyfile or any key
- Only ciphertext crosses `/api/vault`

Your Google Drive

- The vault is one file: `portunus.vault.json`
- OAuth scope is `drive.file`: the app only sees files it created
- Drive gives you sync, history and backups for free

Stateless server

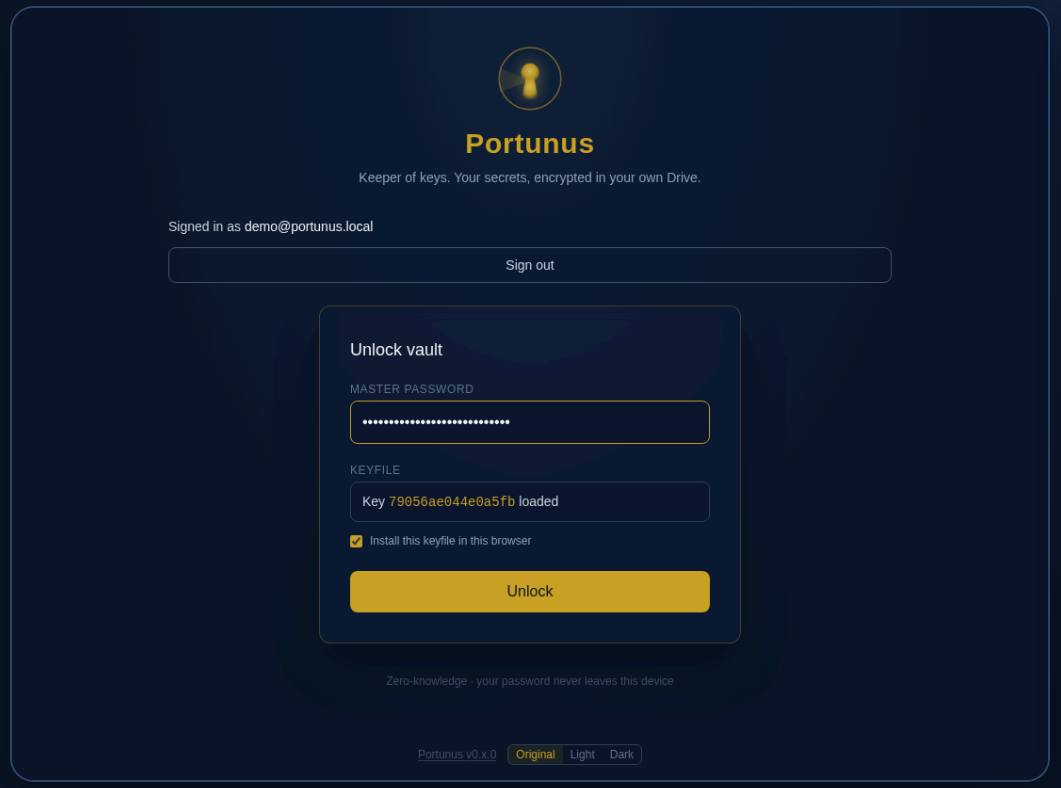
- No database to run, back up, migrate or secure
- Sessions are an encrypted Auth.js JWT cookie
- One standalone, non-root Docker image

The trade-off is deliberate: each user owns their own encrypted file, so there is nothing to share entries *through*. No sharing, no orgs, no admin recovery.

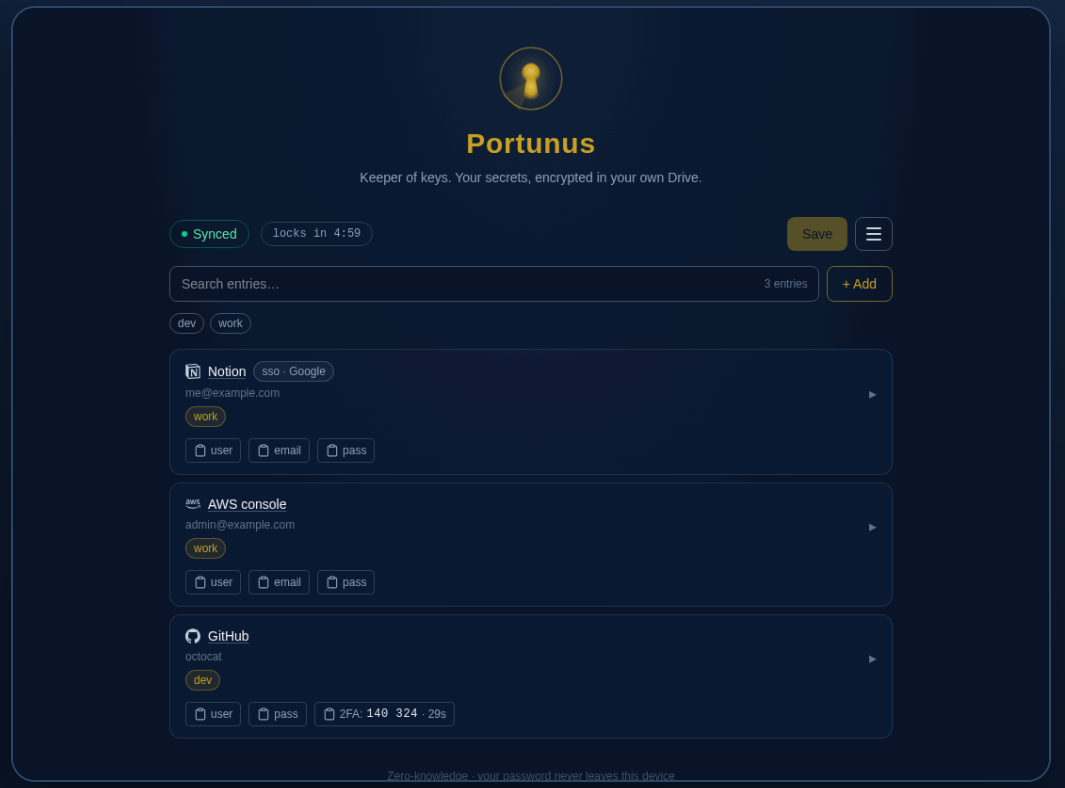
PRODUCT TOUR

What it looks like

Captured from the current build in demo mode.



Unlock: master password + installed keyfile



Vault: search, labels, copy buttons, live 2FA code, auto-lock countdown

FEATURES

What you get

Password manager

- Entries: username/email, password, URL, notes, SSO provider
- Multi-term search and one-click label filters
- Password generator, zxcvbn strength + crack time
- Last 5 passwords kept as history
- Duplicate warnings (same site or reused login)
- ~260 bundled site logos, letter monograms otherwise

Security features

- TOTP: paste a secret, `otpauth://` URI or scan a QR locally
- Have I Been Pwned audits via k-anonymity; hits get a `pwned` label
- Secrets masked; re-masked on blur, tab switch or 30 s idle
- Saving re-asks for the master password
- Drive file signed server-side (Ed25519)

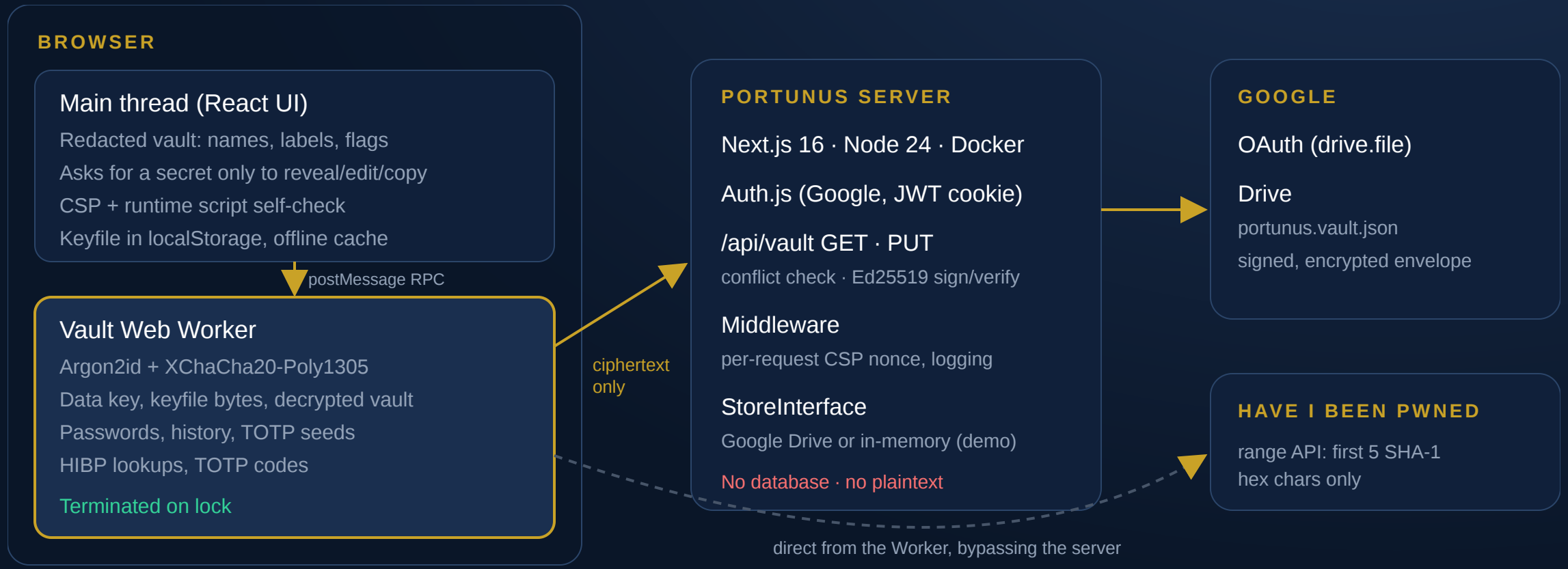
Everyday use

- Auto-lock with a visible countdown; pauses while there are unsaved changes
- Offline mode: read-only from cached ciphertext, recovers on reconnect
- Concurrent-write conflict detection
- Settings synced inside the encrypted vault
- Themes: original navy, light, dark; Focus mode

ARCHITECTURE

Where everything runs

Secrets live in a Web Worker. The server only brokers OAuth and Drive calls on opaque ciphertext.



CRYPTOGRAPHY

How the vault is encrypted

Envelope encryption with a pluggable engine. Engine v1 is `argon2id-xchacha20`.

```
// all of this runs in the vault Worker
KEK      = Argon2id(password || keyfile_bytes, salt)
           m = 19 MiB, t = 2, p = 1 (OWASP)
data_key = 256 random bits
wrapped  = XChaCha20-Poly1305(KEK, data_key)
payload  = XChaCha20-Poly1305(data_key, vault,
             aad = header)

envelope = { format, version, engine, kdf params,
             wrapped key, nonce, ciphertext, website }
```

Properties

- Header fields are AEAD additional data: changing the KDF params, engine id or wrapped key fails decryption
- A fresh random nonce on every save
- Changing the master password re-wraps a fresh data key; the keyfile stays valid
- Vaults from an older engine are re-wrapped under the default engine on the next save
- Libraries: `hash-wasm` (Argon2id) and `libsodium-wrappers`

Pluggable engines: each engine implements `VaultEngineInterface` and registers by id. The envelope records the engine id and its parameters, so a future engine can be introduced without breaking existing vaults.

UNLOCK MODEL

Password and keyfile

Both are mixed into the key derivation. A password alone, on a foreign machine, cannot brute-force the vault.

The keyfile

- 32 random bytes, saved as `portunus.key`
- Its fingerprint (key id) is safe to show in the UI
- Install it in this browser's localStorage, or pick the file each session

The recovery code

- The same key bytes as Crockford-Base32 text
- Shown once at vault creation
- Rebuilds the keyfile on a new device

Creating a vault

- Choose a master password
- Download `portunus.key` and save the recovery code
- Tick the acknowledgement; only then is the vault written to Drive

No password reset, no admin recovery. Losing every keyfile copy *and* the recovery code, or forgetting the password, makes the vault unrecoverable, even for the server operator.

FLOW

Opening the vault

Sign in with Google, then the vault is fetched, checked and decrypted.

1

Sign in

Google OAuth via Auth.js with `drive.file`. Session kept in an encrypted cookie.

→

2

GET /api/vault

Server downloads `portunus.vault.json` from Drive.

→

3

Verify

Ed25519 signature checked. Bad or missing: **422** `vault_signature_invalid`.

→

4

Return payload

Browser gets the envelope, `fileId` and `modifiedTime`. Never the wrapper.

→

5

Decrypt in Worker

Argon2id(password ‖ keyfile), unwrap the data key, decrypt. UI gets a redacted vault.

Offline: if Drive can't be reached, the vault opens **read-only** from the last-synced ciphertext in localStorage. On reconnect it goes back to read-write only if Drive's copy hasn't changed.

Tampered file: the user sees "Vault signature invalid — the file may have been tampered with. Not unlocking." A 422 never falls back to the offline cache.

FLOW

Saving changes

Edits are tracked as unsaved until the user saves. Each save is a full re-encryption.

1. **Edit**
   Changes are made in the Worker. Auto-lock pauses while there are unsaved changes.

→

2. **Confirm**
   User re-enters the master password, checked against the vault header.

→

3. **Encrypt**
   Worker encrypts with a fresh nonce and sends PUT with the loaded `modifiedTime`.

→

4. **Conflict check**
   If Drive's `modifiedTime` differs, **409**. Reload and retry; no overwrite.

→

5. **Sign and write**
   Server signs the envelope with Ed25519 and writes the signed wrapper to Drive.

```
// what Drive holds after a signed save
{ "payload": "{\"format\":\"portunus-vault\", …ciphertext…}",   // exact envelope string
  "signature": "kq3…Zw==" }                                    // base64 Ed25519 over the payload bytes
```

BROWSER HARDENING

Keeping secrets away from page scripts

Web Worker isolation

- Keys, keyfile, decrypted vault, passwords and TOTP seeds stay in `vault.worker.ts`
- The page gets a redacted vault with flags like "has password"
- Secrets are fetched only for an explicit reveal, edit, copy or TOTP action
- Masked fields re-mask on blur, tab switch or 30 s idle

Strict Content Security Policy

- `script-src 'self' 'nonce-…' 'sha256-…'`, no `unsafe-inline` / `unsafe-eval` in production
- Per-request nonce from middleware; inline-script hashes generated at build time
- `object-src 'none'`, `base-uri 'none'`, `frame-ancestors 'none'`
- The primary control: injected scripts are blocked before they run

Runtime script self-check

- Flags `<script>` tags not in the build allowlist, and chunks whose hash changed
- Shows a non-blocking banner: tamper evidence only, best-effort

Known limits

- A script already running in the page can still ask the Worker for secrets
- Malicious browser extensions are out of scope: they run in isolated worlds and can act before the check

STORAGE

Store providers and run modes

```
src/lib/store/
  store-interface.ts      StoreInterface
  providers/
    google-drive-store-provider.ts
    memory-store-provider.ts
  factory.ts              createStoreFactory()

src/factory.ts re-exports every factory;
code imports from @/factory only
```

The same Interface / Provider / Factory pattern is used for every swappable component. Crypto engines implement `VaultEngineInterface` and register by id.

MODE	WHAT CHANGES
Normal	Google OAuth + Drive; signatures enforced
Migration	`PORTUNUS_ALLOW_MISSING_SIGNATURE=1`: legacy unsigned files accepted; each save signs
Signatures off	`PORTUNUS_SIGNATURES_DISABLED=1`: bare envelopes, no key
Demo	`PORTUNUS_DEMO_MODE=1`: stubbed sign-in, in-memory vault per server process, no OAuth or key. Never for real secrets.
No offline	`DISABLE_OFFLINE_MODE=1`: no ciphertext cache

THREAT MODEL

What each party can see

PARTY	CAN SEE	CANNOT SEE
Portunus server	Your Google identity, OAuth tokens, the encrypted envelope, Drive metadata	Plaintext, master password, keyfile, KEK, data key
Google Drive	An encrypted, signed file and its metadata	Anything inside the vault
Have I Been Pwned	The first 5 hex characters of a password's SHA-1	The password or which entry it belongs to
Someone editing Drive	Can replace the file	Can't forge a signature: the server rejects it with 422
Page / main thread	A redacted vault; secrets only when you act on them	Keys and the full decrypted vault (held in the Worker)

Out of scope or not covered: a compromised Portunus server (it holds the signing key and serves the JS); replay of an older, validly signed file; unsigned files during migration; malicious browser extensions. The project has **not** been independently audited; automated Aikido scans are not a substitute.

DEPLOYMENT

Running your own instance

```
# try the UI, no Google setup needed
docker run --rm -p 3000:3000 \
  -e PORTUNUS_DEMO_MODE=1 \
  ghcr.io/ralphv/portunus:latest

# real deployment: docker-compose.yml + .env
AUTH_GOOGLE_ID=…   AUTH_GOOGLE_SECRET=…
AUTH_SECRET=$(openssl rand -hex 32)
AUTH_URL=https://vault.example.com
PORTUNUS_SIGNING_PRIVATE_KEY=base64 PKCS#8 DER
```

- Create your own Google Cloud project and OAuth client, with the `drive.file` scope
- Serve over **HTTPS** on a stable public origin (`AUTH_URL`)
- Generate the Ed25519 key with `openssl genpkey -algorithm ed25519`, convert to one-line base64 DER
- Startup fails fast with a clear error if a required variable or the key is missing or invalid
- Optional: `PORTUNUS_HIBP_ON_EDIT=0` turns off the silent breach re-check on password edits
- The container is stateless: restart, replace or scale it freely

ENGINEERING

How it's built

Stack

- Next.js 16 App Router, React 19
- TypeScript strict, Tailwind CSS v4
- Auth.js v5 (Google provider, JWT)
- hash-wasm, libsodium-wrappers, zxcvbn
- Node `node:crypto` for Ed25519 (server only)
- OpenTelemetry hook at startup

Conventions

- Interfaces end in `Interface`; implementations in `Provider`
- One `factory.ts` per component
- No Node-only APIs in `src/crypto/`
- Test-only files: `*.test.ts`, `*.test-helper.ts`
- README Features list kept in sync per PR

Quality

- Vitest: crypto, keyfile, CSP, script integrity, TOTP, HIBP, worker core, session, signing, API routes
- Round-trip integration test through the real API route
- `npm run build` runs two builds to generate CSP hashes; plain `next build` is refused
- CI: build on Node 20.x / 22.x, plus Aikido security scans

SCOPE AND NEXT STEPS

What it isn't, and what's next

By design, it is not

- A team or family password manager: no sharing between accounts
- Multi-user: no orgs, roles, shared folders or admin recovery
- Recoverable without your keyfile or recovery code
- Independently audited (yet)

Roadmap (TODO.md)

- Archive option for older passwords
- Reminder to clear the clipboard after a few minutes
- Prefill the URL using the same detection as site icons
- Setting to hide other entries when one is opened
- Password generator settings; SSO provider explanation
- Disable and clear the password field when SSO is set
- Fingerprint unlock on mobile

In one line: you own the data, in your own Google account; the server is stateless and the only copy of your secrets is ciphertext in a Drive file you control.